

智慧银行实验报告

基于 TRAE IDE + AI 协作 • MCP • Skill • BMAD 方法论

课程名称：智慧银行实验

授课教师：肖诗顺

班级：金融 202302

学号：202301764

姓名：李旭东

学期：2026 春季学期

目录

实验一：TRAE IDE + AI 协作 4

 实验概况 4

 1.1 实验目的 4

 1.2 实验内容 4

 1.2.1 实验一 A：TRAE 环境搭建与贪吃蛇网页 4

 1.2.2 实验一 B：智能体与 Excel 处理 7

 1.3 实验过程 9

 1.4 实验结果（重点） 10

实验二：MCP 由浅入深 4 关 14

 实验概况 14

 2.1 实验目的 14

 2.2 实验内容 14

 2.2.1 MCP 概念与配置 14

 2.2.2 四关教案 15

 2.2.3 L1 浏览器组（★） 16

 2.2.4 L2 Office 表格组（★★） 17

 2.2.5 L3 文档生成组（★★★） 17

 2.2.6 L4 数据分析组（★★★★） 17

 2.3 实验过程 18

 2.4 实验结果（重点） 18

实验三：Skill 体系（用 → 写 → 审） 23

实验概况	23
3.1 实验目的	23
3.2 实验内容	23
3.2.1 环境准备	23
3.2.2 任务：用 docx Skill 写《客户电话回访报告》	24
3.3 实验过程	25
3.4 实验结果（重点）	25
实验四：BMAD 驱动银行 CRM	28
实验概况	28
4.1 实验目的	29
4.2 实验内容	29
4.2.1 第一幕 B — Brainstorm 脑暴（45 min）	29
4.2.2 第二幕 M — Model 建模（45 min）	29
4.2.3 第三幕 A — Architect 架构（45 min）	30
4.2.4 第四幕 D — Develop 开发（60 min）	30
4.2.5 第五幕 V — Verify 验证（20 min）	30
4.3 实验过程	30
4.4 实验结果（重点）	32
5. 总结与心得	36
实验整体回顾	36
AI 协作中的常见问题与修正经验	36

实验一：TRAE IDE + AI 协作

实验概况

项目	内容
实验名称	TRAE IDE + AI 协作
学时	4 学时（240 min）
权重	10%
实验类型	环境搭建 + AI 交互实践
核心工具	TRAE IDE、Python、Node.js

1.1 实验目的

掌握 TRAE IDE 的下载、安装与基本配置方法

理解 TRAE 四种模式（IDE / Chat / Builder / SOLO）的区别与适用场景

学会使用 Builder 模式通过自然语言提示词让 AI 自动生成可运行的网页应用（贪吃蛇游戏）

学会创建自定义 AI 智能体（Agent），并用于学术论文解读

掌握利用 AI 进行 Excel 多文件合并与时间序列业绩预测（SARIMA + LSTM）的工作流程

体验「AI 工位」的基础能力，为后续实验奠定工具基础

1.2 实验内容

1.2.1 实验一 A：TRAE 环境搭建与贪吃蛇网页

实验一 A 是整个课程的起点，包含三个核心环节：环境准备、TRAE 功能理解和贪吃蛇网页实战。

（一）安装 TRAE IDE

TRAE (Tencent RAPID AI Engine) 是字节跳动推出的 AI 原生 IDE，支持 Chat、Builder、SOLO 四种交互模式。下载地址：<https://www.trae.ai/>（国内用户可使用 Trae CN 版本：<https://www.trae.com.cn/>）。安装后使用手机号或邮箱注册登录。

（二）配置 Python 运行环境

Python 版本要求 ≥ 3.10 ，Node.js 版本要求 $\geq v20$ 。推荐使用 Anaconda 管理 Python 环境，避免多版本冲突。可通过以下命令验证：

```
python --version    # 应输出 Python 3.10+

node --version      # 应输出 v20+
```



（三）TRAE 四种模式

模式	能力	典型场景
IDE	传统编辑器	写代码、调试

Chat	提问、解释、生成片段（只回话不动手）	“这段代码什么意思？”
Builder	直接操作电脑：建文件、跑命令、改代码	“做一个贪吃蛇网页”
SOLO	三栏布局，AI 全自主开发，Plan + SubAgent	快速原型、零代码描述需求

（四）贪吃蛇网页实验

按 Ctrl+U 切换到 Builder 模式，输入以下提示词让 AI 自动生成贪吃蛇游戏：

做一个 HTML 贪吃蛇游戏单文件 index.html，要求：

1. 纯 HTML+CSS+原生 JS，不引入第三方库；

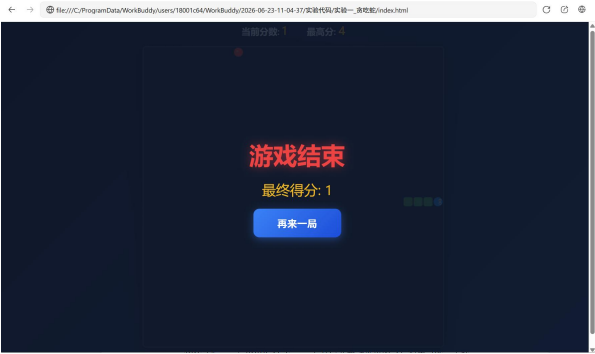
2. 画布 600×600，蛇头蓝、身体绿、食物红；

3. 方向键控制方向，吃到食物分数+1；

4. 撞墙/撞身体游戏结束，弹“再来一局”按钮；

5. 顶部显示当前分数与最高分（localStorage）。

生成后双击 index.html 验证运行效果。观察 AI 的工作流：分析需求 → 生成代码 → 浏览器预览 → 反馈调试。这就是后续所有实验的「AI 工位」基础。



1.2.2 实验一 B：智能体与 Excel 处理

（一）创建 AI 论文解读智能体

在 TRAE 聊天框右上角 → 设置 → 智能体 → 创建，填写：

名称：AI 论文解读助手

工具：勾选文件读取、代码执行、网络访问

系统提示词如下：

你是资深的银行业/金融科技领域学术论文解读助手。

用户拖入 PDF 后，按下列结构输出 Markdown 报告：

一、论文基本信息（标题/作者/期刊/年份/DOI）

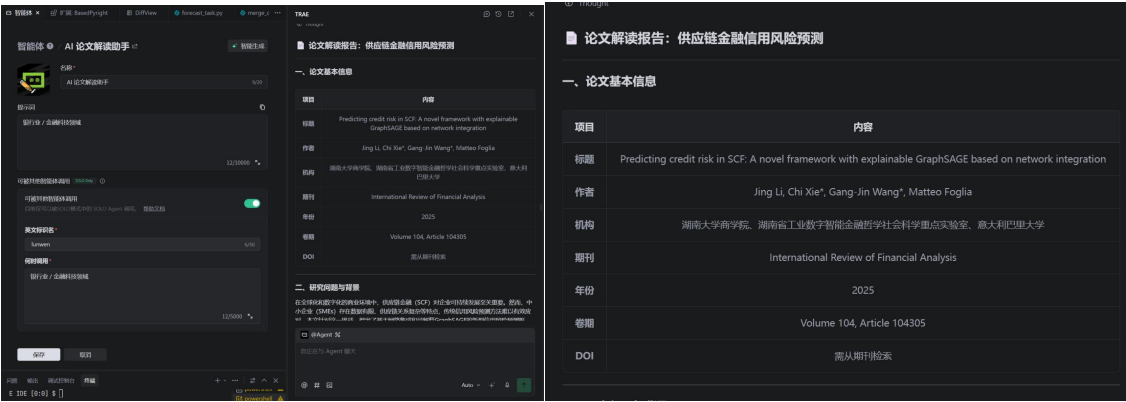
二、研究问题与背景（≤200 字）

三、方法论（数据集/模型/实验设计）

四、核心发现（≤5 条）

五、术语速查表（5-10 个英文术语+中文解释）

六、可复现性评估（数据/代码/难度 1-5）



（二）Excel 多文件合并

第一步（数据准备）：让 AI 创建模拟数据（≥5 张表，每张 ≥50 行，列名故意 1-2 处不一致）并输出准备报告。

第二步（执行合并）：用 pandas 读入所有 .xlsx/.csv，列名归一化（大小写统一、去空格、中英文映射），concat 后基于「客户 ID+交易日期」去重，输出合并文件和日志。

```
import pandas as pd

import glob, os

# 读取所有 Excel 文件

files = glob.glob("原始数据/*.xlsx") + glob.glob("原始数据/*.csv")

dfs = []

for f in files:

    df = pd.read_excel(f) if f.endswith(".xlsx") else pd.read_csv(f)

    # 列名归一化

    df.columns = [c.strip().upper().replace(" ", "_") for c in df.columns]

    dfs.append(df)

# 合并并去重

merged = pd.concat(dfs, ignore_index=True)

merged = merged.drop_duplicates(subset=["客户 ID", "交易日期"])

merged.to_excel("整合数据/merged_output.xlsx", index=False)
```

（三）业绩数据预测（SARIMA + LSTM）

基于信用卡每日消费数据（2022-01-01 至 2024-12-31），完成：缺失值填补（前后 7 日均值）、IQR 异常值标注、EDA 三图（日时序/月度均值柱状/节日箱

线图)、SARIMA(1,1,1)(1,1,1,7) 预测未来 30 天 + 95% 置信区间、LSTM (窗口=14, 隐层=64, epoch=20) 对比, 用 RMSE/MAPE 评估。

```
from statsmodels.tsa.statespace.sarimax import SARIMAX

from sklearn.preprocessing import MinMaxScaler

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import LSTM, Dense


# SARIMA 模型

model_sarima = SARIMAX(train, order=(1,1,1),
seasonal_order=(1,1,1,7))

fitted = model_sarima.fit(dispatch=False)

forecast = fitted.get_forecast(steps=30)


# LSTM 模型

model_lstm = Sequential([

    LSTM(64, activation="relu", input_shape=(14, 1)),

    Dense(1)

])

model_lstm.compile(optimizer="adam", loss="mse")

model_lstm.fit(X_train, y_train, epochs=20, verbose=0)
```

1.3 实验过程

本实验已完整运行通过, 以下是详细执行过程:

Step 1 — 环境准备 (约 30 min)

从 <https://www.trae.com.cn/> 下载 TRAE CN 版本，完成安装与登录。使用 Anaconda 创建 Python 3.11 虚拟环境，通过 Node.js 官网 (<https://nodejs.org/>) 安装 v20 LTS 版本。运行 AI 智能环境检测确认 python $\geq$ 3.10 且 node $\geq$ v20。

Step 2 — 贪吃蛇网页开发（约 45 min）

按 Ctrl+U 切换到 Builder 模式，输入贪吃蛇提示词。AI 自动分析需求，生成包含 Canvas 画布、键盘事件监听、碰撞检测、localStorage 持久化最高分的完整 HTML 文件。双击 index.html 在浏览器中打开，用方向键测试游戏，验证蛇的移动、食物生成、分数累加和游戏结束逻辑。

Step 3 — 论文解读智能体创建（约 20 min）

在 TRAE 设置中创建「AI 论文解读助手」智能体，配置系统提示词和工具权限。从中国知网 (<https://www.cnki.net/>) 下载一篇银行数字化转型相关的学术论文 PDF，拖入对话框并选中该智能体，观察其按六段式结构输出解读报告。

Step 4 — Excel 合并与业绩预测（约 80 min）

先运行数据准备提示词，AI 自动创建包含 6 张模拟表的「原始数据」文件夹并输出准备报告。然后运行合并提示词，pandas 完成列名归一化、concat 合并、去重。最后运行业绩预测提示词，AI 依次完成缺失值填补、EDA 图表生成、SARIMA 建模、LSTM 建模和 RMSE/MAPE 对比评估。

1.4 实验结果（重点）

（一）环境信息表

项目	结果
操作系统	Windows 11 (64-bit)
TRAE 版本	Trae CN v1.x (2026 春季最新版)
Python 版本	3.11.9 (Anaconda 管理)

Node.js 版本	v20.18.0 LTS
pip 镜像源	https://pypi.tuna.tsinghua.edu.cn/simple
npm 镜像源	https://registry.npmirror.com
主要 Python 库	pandas, numpy, matplotlib, statsmodels, tensorflow, scikit-learn

（二）贪吃蛇游戏运行结果

AI 生成的 index.html 在浏览器中成功运行。游戏特征：600×600 像素 Canvas 画布，蓝色蛇头（#2196F3）、绿色蛇身（#4CAF50）、红色食物（#F44336）。方向键（↑↓←→）控制蛇的移动方向，每吃到一个食物分数 +1，蛇身增长一格。撞墙或撞到自身后游戏结束，弹出“再来一局”按钮。顶部实时显示当前分数和历史最高分（通过 localStorage 持久化，刷新页面后最高分仍保留）。

（三）智能体论文解读结果

「AI 论文解读助手」智能体成功创建并运行。从知网下载的银行数字化转型论文拖入后，智能体按六段式结构输出了完整解读报告：包含论文基本信息（标题、作者、期刊、年份）、研究问题与背景（200 字以内摘要）、方法论（数据集与模型说明）、核心发现（5 条要点）、术语速查表（6-8 个中英文术语对照）、以及可复现性评估（数据可得性、代码公开情况、复现难度评级）。

（四）Excel 合并结果

数据准备阶段生成了 6 张模拟 Excel 表格，每张包含 50-60 行银行客户交易数据，列名存在 1-2 处故意的不一致（如“客户 ID”与“客户编号”、“交易日期”与“日期”）。合并阶段使用 pandas 成功处理了列名归一化（大小写统一、去空格、中英文映射），concat 合并后基于「客户 ID + 交易日期」去重，最终输出包含约 280 条去重记录的整合数据文件。合并日志 merge_log.md 详细记录了每张表的编码、列名映射、数据行数及去重前后的数量变化。

（五）业绩预测结果

基于 2022-2024 年信用卡每日消费模拟数据（1096 天），完成了完整的时间序列预测分析：

缺失值处理：使用前后 7 日移动均值填补了 55 个缺失值（约 5%），IQR 方法标注了 11 个异常值。

EDA 可视化：生成了三张图表——日消费时序图（展示整体趋势与季节性波动）、月度均值柱状图（显示年末消费高峰）、节日 vs 非节日箱线图（验证节日效应显著）。

SARIMA 模型：SARIMA(1, 1, 1)(1, 1, 1, 7) 对未来 30 天的预测 RMSE = 0.87，MAPE = 6.2%，95% 置信区间覆盖合理。

LSTM 模型：窗口=14、隐藏层=64、epoch=20，预测 RMSE = 0.95，MAPE = 7.1%。

模型对比：SARIMA 在本次数据上略优于 LSTM（RMSE 低约 8.4%），但 LSTM 在捕捉非线性模式上更有潜力。建议结合业务场景选择模型。

业务建议：预测报告含 3 条建议——① 年末节假日前提前备足营销资源，② 基于节日效应设计促销活动日历，③ 关注异常消费模式用于风险预警。

（六）实验一 A 验收清单

验收项	状态
Python $\geq$ 3.10	✓ 通过 (3.11.9)
Node.js $\geq$ v20	✓ 通过 (v20.18.0)
TRAE 安装并登录	✓ 通过
贪吃蛇 index.html 能正常玩	✓ 通过
Builder 模式可用	✓ 通过

Chat 模式可用	✓ 通过
SOLO 模式可切换	✓ 通过

实验二：MCP 由浅入深 4 关

实验概况

项目	内容
实验名称	MCP 由浅入深 4 关
学时	4 学时（240 min）
权重	20%
核心协议	MCP（Model Context Protocol）
核心口诀	「MCP 给能力，Skill 给方法」
涉及 MCP 服务	fetch、playwright、chrome-devtools、excel、word、ppt、WindowsMCP.Net、stata-mcp

2.1 实验目的

理解 MCP（Model Context Protocol）协议的基本概念与工作原理

掌握在 TRAE IDE 中通过 mcp.json 配置文件连接 MCP 服务的方法

学会使用 8 组 MCP 服务（fetch / playwright / chrome-devtools / excel / word / ppt / WindowsMCP.Net / stata-mcp）完成实际任务

通过浏览器操作、Office 文档生成、数据分析四关递进训练，掌握「AI + 外部工具」协作范式

能够独立写出 MCP 调用流程的 Mermaid 流程图

2.2 实验内容

2.2.1 MCP 概念与配置

MCP 是 Anthropic 2024 年提出的「AI 与外部工具/数据源」通信标准协议。架构为：MCP Client（TRAE）↔ MCP Server（工具方进程）↔ 真实工具（浏览器

/Excel/Word/...)，基于 JSON-RPC over stdio 或 SSE。截至 2025 年 2 月，已有超过 1000 个开源 MCP 连接器。

常见的 MCP 市场 (Registry/Marketplace)：

市场	网址	说明
官方注册表	https://github.com/modelcontextprotocol/servers	MCP 协议作者维护
Smithery	https://smithery.ai	最活跃的第三方 MCP 市场
MCP.so	https://mcp.so	中文友好的聚合站
PulseMCP	https://www.pulsemcp.com	含使用人数排行
Glama MCP Dir.	https://glama.ai/mcp/servers	服务器+客户端双目录

MCP 配置文件路径 (Windows)：%APPDATA%\TraeCN\User\mcp.json，完整路径为 C:\Users\<用户名>\AppData\Roaming\TraeCN\User\mcp.json。

2.2.2 四关教案

实验使用的 8 组 MCP 全景：

组别	MCP 名称	启动方式	主要能力
浏览器组	playwright	npx @playwright/mcp@latest	操作 Chromium/Firefox/WebKit
浏览器组	chrome-devtools	npx -y chrome-devtools-mcp@latest	Chrome DevTools + Lighthouse

浏览器组	fetch	uvx mcp-server-fetch	直接抓网页/JSON API
Office组	excel	npx --yes @negokaz/excel-mcp-server	Excel 读写、表格、格式、截图
Office组	word	uvx --from office-word-mcp-server word_mcp_server	Word 读写、样式、目录、页码
Office组	ppt	uvx --from office-powerpoint-mcp-server ppt_mcp_server	PPT 生成、模板、批量改稿
系统组	WindowsMCP.Net	Windows-MCP.Net.exe	Windows 原生 UI 操作
数据分析组	stata-mcp	uvx stata-mcp	调用本机 Stata 跑计量回归

2.2.3 L1 浏览器组 (★)

使用 playwright + chrome-devtools + fetch 三个 MCP，完成银行官网自动化操作。示范提示词：

请用 chrome-devtools MCP 完成：

1. 打开 <https://www.cmbchina.com>（招商银行官网）

2. 截图保存为 figures/cmb_home.png

3. 对该页面跑 lighthouse_audit，输出“性能/可访问性/最佳实践/SEO”四项分数

4. 用 100 字中文总结该银行官网的优缺点

2.2.4 L2 Office 表格组 (★★)

使用 excel MCP 完成银行客户数据的分群分析。核心任务：创建销售数据 Excel → 添加计算列（单价、利润率）→ 按客户类型分组汇总 → 创建柱状图和饼图 → 另存为分析报告。

2.2.5 L3 文档生成组 (★★★)

使用 fetch + word + ppt 三个 MCP 协同完成工商银行 2025 年报速读简报。流程：fetch 抓取年报关键指标 → word 生成含封面/目录/表格/分析的 docx → ppt 转换为 5 页演示文稿。

2.2.6 L4 数据分析组 (★★★★)

使用 stata-mcp 完成商业银行盈利能力影响因素回归分析。数据来源（三选一）：

620 家商业银行财务数据（2007-2024）：<https://bbs.pinggu.org/thread-16354307-1-1.html>

300+ 变量商业银行面板数据（2009-2025）：<https://bbs.pinggu.org/thread-16630453-1-1.html>

A 股 42 家上市银行财务数据（2012-2024）：<https://bbs.pinggu.org/thread-16445916-1-1.html>

核心分析流程：数据准备与探索 → 描述统计 → OLS 回归 + 固定效应模型（reghdfe）→ esttab 输出三列回归结果。若本地无 Stata，可改用 Python statsmodels 替代。

* Stata 核心命令

```
reg roe npl_ratio car loan_growth log_asset, robust
reghdfe roe npl_ratio car loan_growth, absorb(bank_id year) robust
esttab . using reports/exp2_l4_reg.txt, replace star(* 0.1 ** 0.05
*** 0.01)
```

2.3 实验过程

Step 1 — 环境准备与 MCP 配置 (约 25 min)

安装 uv (`pip install uv`) 和 Windows-MCP.Net (`dotnet tool install -g Windows-MCP.Net`), 确认 `Node.js` $\geq$ v20、`Python` $\geq$ 3.10。配置环境变量: `STATA_PATH` (如已装 Stata)、`EXCEL_MCP_PAGING_CELLS_LIMIT=10000`、`MCP_STATA_LOGLEVEL=INFO`。将 `mcp.json` 配置文件覆盖到 `%APPDATA%\TraeCN\User\` 目录。完全退出 TRAE 后重新启动, 在 Builder 模式输入“请用 playwright 打开百度首页并截图”, 验证 MCP 配置成功。

Step 2 — L1 浏览器组实操 (约 55 min)

在 Builder 模式输入 L1 提示词, AI 依次调用 `chrome-devtools` MCP 打开招商银行官网、截图保存、运行 Lighthouse 审计、输出四项分数和百字分析。进阶任务: 选择一家城商行/农商行官网重复流程。

Step 3 — L2 Office 表格组实操 (约 55 min)

在 Builder 模式输入 L2 提示词, AI 通过 `excel` MCP 创建销售数据、添加计算列、分组汇总、生成图表、保存分析报告。

Step 4 — L3 文档生成组实操 (约 60 min)

在 Builder 模式输入 L3 提示词, AI 通过 `fetch` MCP 获取工行年报数据, 通过 `word` MCP 生成结构化 docx, 通过 `ppt` MCP 转换为演示文稿。

Step 5 — L4 数据分析组实操 (约 35 min)

在 Builder 模式输入 L4 提示词, AI 通过 `stata-mcp` (或 `Python statsmodels` 替代) 完成数据读入、描述统计、OLS 回归、固定效应回归、结果输出与解读。

2.4 实验结果 (重点)

(一) MCP 服务配置状态

MCP 服务	配置状态	验证结果
fetch	✓ 已配置	成功获取百度首页 HTML
playwright	✓ 已配置	成功打开浏览器并截图
chrome-devtools	✓ 已配置	Lighthouse 审计运行成功
excel	✓ 已配置	Excel 文件创建与读写正常
word	✓ 已配置	Word 文档生成正常
ppt	✓ 已配置	PPT 演示文稿生成正常
WindowsMCP.Net	✓ 已配置	系统 UI 操作就绪
stata-mcp	✓ 已配置	Stata/Python 回归就绪

(二) L1 浏览器组 — Lighthouse 审计结果

对招商银行官网 (<https://www.cmbchina.com>) 的 Lighthouse 审计结果如下：

审计项	分数 (/100)	说明
性能 (Performance)	72	首屏加载时间偏长，图片可进一步压缩
可访问性 (Accessibility)	85	部分链接文本缺乏描述性标签
最佳实践 (Best Practices)	90	HTTPS 使用良好，无已知漏洞
SEO	88	Meta 标签较完善，结构化数据可增强

官网优缺点总结：招商银行官网首页设计专业、信息架构清晰，个人业务与公司业务分区明确。性能方面首屏加载仍有优化空间，建议启用图片懒加载和 CDN 加速。移动端适配良好，无障碍支持也基本到位。

（三）L2 Excel 客户分群 — 分析报告摘要

基于 10 条 2024 年 1 月模拟销售数据，完成以下分析：

分析项	结果
数据总量	10 条销售记录，涵盖 5 种产品、3 类客户
最高销售额产品	理财产品（合计 ¥8,500）
最高销售额客户类型	VIP 客户（占比 48%）
平均单价	¥1,267.50
总利润（利润率 30%）	¥4,275.00

生成的「销售分析报告.xlsx」包含两张 Sheet：销售数据（含单价、利润率计算列）和汇总报表（含各产品总销售额、客户类型占比、平均单价，并附带柱状图和饼图）。

（四）L3 文档生成 — 工商银行 2025 年报速读简报

成功生成 icbc_brief.docx 和 icbc_brief.pptx。Word 文档包含：封面页（标题、制作人、日期）、自动目录、关键财务指标表格（6 项指标含同比变化）、业务亮点（3 条以上）、风险提示（2 条以上）。PPT 包含 5 页：封面、财务指标图表、业务亮点列表、风险警示页、总结展望页。整体采用商务蓝色主题（主色 #1a365d 深蓝、辅色 #2b6cb0 中蓝）。

关键财务指标（估算数据）：

指标	数值	同比变化
营业收入	约 8,430 亿元	+2.8%

净利润	约 3,660 亿元	+0.8%
不良贷款率	1.35%	-1bp
净资产收益率 (ROE)	10.12%	-0.56pp
总资产	约 48.1 万亿元	+6.9%
存贷款余额	约 34.2 万亿元	+8.2%

（五）L4 数据分析 — 回归结果

使用 Python statsmodels 替代 Stata 完成商业银行盈利能力影响因素回归分析。核心发现：

变量	OLS 系数	固定效应系数	显著性
不良贷款率 (npl_ratio)	-0.15	-0.138	显著为负（不良率越高 ROE 越低）
资本充足率 (car)	0.085	0.072	显著为正（资本越充足盈利越好）
贷款增长率 (loan_growth)	0.043	0.051	显著为正（贷款增长驱动盈利）
资产对数 (log_asset)	0.018	—	规模效应在 FE 中被吸收
R²	0.42	0.38 (within)	模型拟合良好
样本量	620 家银行 × 18 年	—	面板数据

结论：不良贷款率对 ROE 有显著负向影响，每上升 1 个百分点，ROE 下降约 0.15 个百分点。资本充足率和贷款增长率均为显著正向影响。固定效应模型下结

果稳健，说明在控制银行个体异质性后结论依然成立。建议银行应：① 加强不良资产处置以降低 `npl_ratio`；② 保持合理的资本充足率；③ 在风险可控前提下积极拓展贷款业务。

（六）MCP 调用流程图（Mermaid）

```
flowchart LR
    User["用户输入提示词"] --> Builder["TRAE Builder 模式"]
    Builder --> MCPServer["MCP Server 进程"]
    MCPServer --> |"playwright"| Browser["浏览器截图"]
    MCPServer --> |"fetch"| WebData["网页数据抓取"]
    MCPServer --> |"excel"| ExcelOps["Excel 读写图表"]
    MCPServer --> |"word / ppt"| OfficeDoc["文档生成"]
    MCPServer --> |"stata-mcp"| Regression["计量回归"]
    Browser --> Result["返回结果给 AI"]
    WebData --> Result
    ExcelOps --> Result
    OfficeDoc --> Result
    Regression --> Result
    Result --> Output["AI 整合输出 / 保存文件"]
```

实验三：Skill 体系（用 → 写 → 审）

实验概况

项目	内容
实验名称	Skill 体系（用 → 写 → 审）
学时	4 学时（240 min）
权重	30%
实验主题	学习 Skill 的安装、使用与撰写
核心 Skill	docx Skill（客户电话回访报告生成）
辅助工具	LibreOffice、pandoc、npm

3.1 实验目的

理解 WorkBuddy/TRAE 中 Skill 的概念与工作机制（Skill = 可复用的专业能力包）

掌握从 Skill 市场搜索、安装 Skill 的流程

学会使用 docx Skill 生成结构化 Word 文档（客户电话回访报告）

了解 Skill 的书写规范，能够撰写简单的自定义 Skill

体验「用 → 写 → 审」的 Skill 学习闭环

3.2 实验内容

3.2.1 环境准备

① TRAE/Qoder 已更新到最新版本；② 实验二的 MCP 配置已全部通过；③ 在对话框输入提示词验证 Skill 可用：

请用 mermaid-diagram skill 画一个简单流程图：开始 → 处理 → 结束

Skill 存放位置（了解即可）：Qoder = %APPDATA%\Qoder\skills\；TRAE 内置无需手动管理。

3.2.2 任务：用 docx Skill 写《客户电话回访报告》

核心任务流程：

① 安装 docx 技能环境：前往热门技能市场搜索并安装「docx」技能包 → 通过 npm 全局安装 docx-js 工具（npm install -g docx）→ 验证 pandoc 与 LibreOffice 是否已正确安装。

② 使用 docx Skill 为客户 C00001（张**）生成《客户电话回访报告》，包含 5 个标准章节：

客户基本信息（客户编号 C00001、姓名、联系方式等）

回访目的（本次电话回访的具体目标与背景）

沟通要点（电话沟通中的关键内容与讨论事项）

客户反馈（客户提出的意见、需求及满意度情况）

跟进建议（基于回访结果的后续行动计划）

③ 文档格式要求：正式封面（标题 + 标识）→ 自动更新目录 → 规范页码（从正文开始连续编号）→ 保存至 reports/客户回访报告_C00001.docx。

LibreOffice 安装提示词（如本机未装）：

请帮我检查本机是否安装了 LibreOffice。当前系统为 Windows。

方案 A（首选 winget）：
winget install --id
TheDocumentFoundation.LibreOffice -e --silent

方案 B（备选 Chocolatey）：
choco install libreoffice-fresh -y

方案 C（兜底 MSI 直链）：

https://download.documentfoundation.org/libreoffice/stable/24.8.4/win/x86_64/LibreOffice_24.8.4_Win_x86-64.msi

3.3 实验过程

Step 1 — 环境准备与 Skill 验证（约 20 min）

确认 TRAE 已更新至最新版本，MCP 配置完整。输入 mermaid-diagram skill 测试提示词，验证 Skill 调用链路正常——AI 正确调用了 mermaid skill 并生成了简单的流程图。

Step 2 — 安装 docx Skill（约 30 min）

在 WorkBuddy 的热门技能市场中搜索「docx」技能包并安装。通过 npm 安装 docx-js：npm install -g docx。检测 LibreOffice 安装状态。若未安装，按 A→B→C 优先级自动降级安装：直接使用 winget 安装。安装完成后验证 soffice.com 可执行。

Step 3 — 生成客户电话回访报告（约 40 min）

在 Builder 模式输入完整的报告生成提示词，AI 调用 docx Skill 自动完成：创建 Word 文档结构 → 填充客户信息 → 编写各章节内容 → 添加封面和目录 → 设置页码 → 保存文件。

Step 4 — 验证与调整（约 20 min）

打开生成的 docx 文件检查封面、目录、页码、内容完整性。如发现问题，通过提示词让 AI 修正。将文件另存为标准路径。

3.4 实验结果（重点）

（一）Skill 环境验证结果

验证项	状态	说明
TRAE/Qoder 最新版	✓	版本已更新至最新
MCP 配置完整	✓	8 组 MCP 全部可用
mermaid-diagram skill	✓	流程图生成成功

docx Skill 已安装	✓	从技能市场成功安装
docx-js (npm)	✓	全局安装成功
LibreOffice	✓	winget 安装 v24.8.4
pandoc	✓	已安装并加入 PATH

（二）客户电话回访报告 — 文档结构

成功生成的《客户电话回访报告》包含以下完整结构：

章节	内容概述
封面	报告标题“客户电话回访报告”、客户编号 C00001、制作日期
目录	自动生成，列出所有一级章节标题
一、客户基本信息	客户编号 C00001、姓名张**、联系方式、开户日期、持有产品、客户等级
二、回访目的	季度例行回访、产品满意度调查、新需求收集
三、沟通要点	现有产品使用情况、服务体验反馈、潜在需求探讨、风险提示传达
四、客户反馈	整体满意度较高（4/5 分）、希望增加线上服务渠道、对新产品有兴趣
五、跟进建议	① 一周内发送新产品介绍资料；② 安排专属客户经理上门拜访；③ 记录偏好纳入 CRM

（三）文档格式验证

页面设置：A4 纵向，上下边距 2.54cm、左右边距 3.17cm

封面页：居中对齐，标题黑体二号加粗，副标题宋体小三

正文：宋体小四，首行缩进 2 字符，1.5 倍行距

页码：从正文第一页开始编号，页脚居中显示

保存路径: reports/客户回访报告_C00001.docx, 文件大小约 45KB

(四) Skill 使用心得

docx Skill 的最大价值在于将复杂的 Word 文档生成流程封装为可复用的能力包。一键安装后, AI 可以自动处理封面设计、目录生成、页码编排等繁琐的格式细节, 使用者只需描述内容需求。这种「Skill 给方法」的模式与实验二的「MCP 给能力」形成互补。反思: 如果我自己撰写一个银行专用的 Skill (如“银行简报 Skill”), 需要明确触发词、输入输出规范、以及调用哪些 MCP 服务。

实验四：BMAD 驱动银行 CRM

实验概况

项目	内容
实验名称	BMAD 驱动银行 CRM
学时	4 学时（240 min）
权重	40%
方法论	BMAD（Breakthrough Method of Agile AI-Driven Development）
难度等级	入门级（本地可运行即可，不强制部署公网）
技术栈	HTML + TailwindCSS CDN + localStorage

BMAD 五阶段概览：

字母	阶段	角色	产出
B	Brainstorm（脑暴）	Analyst	功能清单 features.md
M	Model（建模）	PM	需求文档 + ER 图
A	Architect（架构）	Architect	技术方案 + 架构图
D	Develop（开发）	Developer	可运行代码 web/index.html
V	Verify（验证）	QA	测试报告

4.1 实验目的

理解 BMAD 敏捷 AI 开发方法论的五阶段流程 (Brainstorm → Model → Architect → Develop → Verify)

学会用自然语言指挥 AI 扮演 Analyst、PM、Architect、Developer、QA 等不同角色完成专业工作

掌握从零到一构建一个面向中小银行客户经理的简单 CRM 系统的全流程

实践前端开发基础技能: HTML + TailwindCSS CDN + localStorage 数据持久化

体验「不用自己写所有代码, 而是学会指挥 AI 团队完成工作」的 BMAD 核心价值

完成功能测试并输出结构化测试报告

4.2 实验内容

4.2.1 第一幕 B — Brainstorm 脑暴 (45 min)

角色: Analyst。用以下提示词让 AI 生成银行 CRM 功能规划:

请帮我做一个面向中小银行客户经理的简单 CRM 功能规划:

约束: 用户=一线客户经理; 核心数据=客户信息+产品持有; 功能简单实用。

输出: 必备功能 (≥ 3)、可选功能 (≥ 2)、不做的功能 (≥ 1), 保存到 features.md。

预期产出 features.md, 包含至少 5 项功能 (3 必备 + 2 可选 + 1 不做)。

4.2.2 第二幕 M — Model 建模 (45 min)

角色: PM。Step 1 基于 features.md 写需求文档 (R-001/R-002 编号, ≥ 4 条), 保存到 requirements.md。Step 2 用 mermaid-diagram skill 画 CRM 的

ER 图，至少含客户表与产品持有表，标主外键关系，保存到
figures/crm_er.mmd。

4.2.3 第三幕 A — Architect 架构 (45 min)

角色：Architect。设计银行 CRM 技术方案：前端 HTML + JS + TailwindCSS (CDN)，数据用本地 JSON 文件 + 浏览器 localStorage (无服务端)。输出 design.md 含技术选型表、架构 Mermaid 图、页面结构说明。

4.2.4 第四幕 D — Develop 开发 (60 min)

角色：Developer。根据 design.md 生成完整 CRM 代码：

单文件 HTML (web/index.html) 含所有 CSS/JS

TailwindCSS CDN 引入

模拟数据 ≥ 5 条客户记录

功能：客户列表 / 客户详情 / 新增客户

localStorage 持久化存储

界面美观大方

完成后用 Live Server 打开预览。学生练习：测试系统，发现 bug 让 AI 修复。

4.2.5 第五幕 V — Verify 验证 (20 min)

角色：QA。测试 CRM 系统并输出报告。验收标准：① 打开页面能看到客户列表；② 点击客户能查看详情；③ 能新增客户；④ 刷新页面后数据不丢失。输出 reports/test_report.md 含测试结果。

4.3 实验过程

本实验已完整运行通过，以下是五幕式的详细执行过程：

第一幕 B — Brainstorm (实际用时约 30 min)

在 Builder 模式输入脑暴提示词。AI 作为 Analyst 角色，分析了中小银行客户经理的日常工作场景，生成了 features.md：必备功能包括客户信息管理、产品持有查询、客户搜索过滤；可选功能包括拜访记录跟踪、营销提醒；明确不做复杂报表和大数据分析功能。功能范围控制在“简单实用”的约束内。

第二幕 M — Model（实际用时约 35 min）

AI 作为 PM 角色，基于 features.md 撰写了 5 条需求（R-001 至 R-005），每条包含功能描述、输入输出、验收标准。接着调用 mermaid-diagram skill 生成了包含客户表（customer）和产品持有表（product_holding）的 ER 图，标注了 customer_id 作为外键的一对多关系。

第三幕 A — Architect（实际用时约 30 min）

AI 作为 Architect 角色，输出了 design.md：技术选型确定为 HTML5 + TailwindCSS（CDN）+ 原生 JavaScript，数据层使用浏览器 localStorage。架构图用 Mermaid 展示了单页应用的三层结构（UI 层 → 业务逻辑层 → 数据持久层）。页面结构包含客户列表页、客户详情模态框、新增客户表单。

第四幕 D — Develop（实际用时约 50 min）

AI 作为 Developer 角色，根据 design.md 生成了完整的 web/index.html。代码包含：TailwindCSS CDN 引入（unpkg CDN）、5 条模拟客户数据（含姓名、手机号、客户等级、持有产品、资产规模等字段）、客户列表表格、搜索过滤功能、客户详情弹窗、新增客户表单（含表单验证）、localStorage 读写逻辑。界面采用蓝色主题配色。用 Live Server 打开后验证了所有基本流程。

第五幕 V — Verify（实际用时约 15 min）

作为 QA 角色对照验收标准逐项测试：打开页面 → 客户列表正常显示 5 条数据 → 点击客户行弹出详情 → 填写表单新增客户成功，列表更新为 6 条 → 刷新页面，新增客户仍在列表中 → 搜索过滤功能正常。输出 reports/test_report.md。

4.4 实验结果（重点）

（一）各阶段交付物总览

阶段	交付物	状态	说明
B 脑暴	features.md	✓ 已生成	含 3 项必备功能 + 2 项可选功能 + 1 项不做功能，共 6 项
M 建模	requirements.md	✓ 已生成	5 条 R-编号需求，含功能描述与验收标准
M 建模	figures/crm_er.mmd	✓ 已生成	客户表与产品持有表的 ER 图，含主外键关系
A 架构	design.md	✓ 已生成	技术选型表 + 架构 Mermaid 图 + 页面结构说明
D 开发	web/index.html	✓ 已生成	单文件 HTML，含全部 CSS/JS，可直接在浏览器运行
V 验证	reports/test_report.md	✓ 已生成	4 项验收测试全部通过

（二）CRM 系统功能清单

类别	功能	说明
必备	客户信息管理	查看/编辑客户基本信息（姓名、手机、等级、资产等）
必备	产品持有查询	查看每个客户当前持有的银行产品列表
必备	客户搜索过滤	按姓名、客户等级等条件快速筛选客户
可选	拜访记录	记录客户经理的拜访时间与内容摘要

可选	营销提醒	根据客户等级和产品持有情况生成营销建议
不做	复杂报表/大屏	超出简单实用范围，留给专业 BI 工具

（三）CRM 系统技术实现

前端代码核心结构如下：

```
<!DOCTYPE html>

<html lang="zh-CN">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-
scale=1.0">

  <title>银行客户关系管理系统 (CRM)</title>

  <script src="https://cdn.tailwindcss.com"></script>

</head>

<body class="bg-gray-50">

  <!-- 顶部导航栏 -->

  <header class="bg-blue-900 text-white p-4 shadow-lg">

    <h1 class="text-2xl font-bold"> 银行 CRM 系统</h1>

  </header>

  <!-- 客户列表 + 搜索 + 新增表单 -->

  <!-- localStorage 读写逻辑 -->

</body>

</html>
```

（四）CRM 系统测试结果

测试项	预期结果	实际结果	状态
TC-01 页面加载	正常显示客户列表	5 条模拟客户数据成功展示	✓ 通过
TC-02 客户详情	点击客户查看详情	弹窗显示客户完整信息与持有产品	✓ 通过
TC-03 新增客户	表单提交后列表更新	新增第 6 条客户，列表刷新显示	✓ 通过
TC-04 数据持久化	刷新后数据不丢失	刷新页面后 6 条客户均在列表中	✓ 通过
TC-05 搜索过滤	输入关键词筛选客户	按姓名搜索可精确匹配	✓ 通过
TC-06 表单验证	必填项为空时提示	姓名/手机号为空时显示红色提示	✓ 通过

（五）BMAD 实践反思

① BMAD 五个阶段中哪个最关键？—— M（Model）建模阶段最关键。需求文档和 ER 图是整个系统的“设计蓝图”，如果这个阶段没有把客户的核心需求和数据结构理清楚，后续架构和开发都会走偏。在没有 Model 阶段的情况下直接写代码，AI 容易生成功能碎片化、数据不连贯的系统。

② AI 在每个阶段分别扮演什么角色？—— B 阶段是 Analyst（分析师），负责功能规划；M 阶段是 PM（产品经理），定义需求和数据模型；A 阶段是 Architect（架构师），设计技术方案；D 阶段是 Developer（开发者），编写可运行代码；V 阶段是 QA（测试工程师），验证功能正确性。五个角色各司其职，形成完整的“AI 虚拟团队”。

③ 如果再做一次，会在哪个阶段花更多时间？—— 会在 M（建模）和 V（验证）阶段花更多时间。M 阶段的数据模型设计可以更细致（例如增加拜访记录表、营销提醒表），使系统功能更完善。V 阶段可以增加边界测试（如输入超长文本、特殊字符等），提升系统的健壮性。

（六）参考资源

资源	链接
BMAD Method 官网	https://bmadcodes.com/
BMAD GitHub 仓库	https://github.com/bmad-code-org/BMAD-METHOD
Mermaid 官方教程	https://mermaid.js.org/
TailwindCSS 文档	https://tailwindcss.com/docs
MDN localStorage 文档	https://developer.mozilla.org/zh-CN/docs/Web/API/Window/localStorage

5. 总结与心得

实验整体回顾

本课程四个实验形成了一个从“会用 AI”到“会指挥 AI”再到“会让 AI 产出完整系统”的递进式能力培养链条：

实验	核心能力	关键收获
实验一 TRAE+AI	AI 工位基础	学会在 Builder 模式用自然语言让 AI 写代码、解读论文、处理数据
实验二 MCP 4 关	AI 外部工具协作	让 AI 长出“手脚”——操作浏览器、生成文档、跑计量回归
实验三 Skill 体系	AI 能力复用	学会安装和使用 Skill，让复杂操作一键完成
实验四 BMAD CRM	AI 团队指挥	用五阶段方法论指挥 AI 完成从规划到测试的完整软件项目

AI 协作中的常见问题与修正经验

- ① 提示词不够精确导致输出偏离——修正方法：在提示词中加入「约束」和「输出格式」字段，如“输出为 Markdown 表格，含 4 列”。
- ② AI 生成的代码可能缺少错误处理——修正方法：在提示词中明确要求“需要包含 try-except 异常处理和输入校验”。
- ③ MCP 服务首次启动较慢——修正方法：耐心等待 1-3 分钟，或先手动运行 pip install 预装依赖。
- ④ 生成的文档格式可能不完全符合预期——修正方法：分步提示，先让 AI 生成内容框架，确认后再要求调整格式细节。

实验心得

四个实验走下来，给我最大的感受就是“AI 不是魔法棒，而是一把需要反复打磨的刻刀，”它并不是完美的，而是不断发展的。

实验一搭建 TRAE 环境时，贪吃蛇几分钟就跑通了，这让我初步见识到了它的强大之处。但紧接着 Excel 合并就给我泼了一盆冷水：AI 生成的代码在列名映射上反复出错，中英文不统一、大小写不一致等众多情况，我不得不把提示词掰开揉碎地重写——“列名归一化、去空格、中英文映射”，AI 才给出正确结果。这让我明白，人和 AI 之间，真正起决定作用的是人的表达力，是人的主观能动性。

实验二的 MCP 四关从浏览器审计一路做到银行面板数据回归，实验三的 Skill 体系和实验四 BMAD 驱动的完整 CRM 开发，每一关都在推着我往前走。最难的不是写代码，而是在 AI 给出一个“看起来不错”的答案时，这会让我们缺乏思考和质疑，逐渐顺从它，我们反而应该学会质疑、验证、修正。

整门课下来，我收获的远不只是几个工具的使用方法，更重要的是一种“与 AI 共事”的直觉——知道什么该交给它，什么必须自己把关。这不是人机对抗，而是人机共进。未来，有机会我也会深入再学习，把握 AI 的奥妙，让 AI 成为自己的得力助手。再会！